

PRACTICAL 6

AIM:

Implementation of chain matrix multiplication using dynamic programming.

ALGORITHM:

Algorithm.

1. For $i = 1$ to n do
2. $M[i][i] \leftarrow 0$
3. For $L = 2$ to n do
4. For $i = 1$ to $n - L + 1$ do
5. $j \leftarrow i + L - 1$
6. $M[i][j] \leftarrow \infty$
7. For $k = i$ to $j - 1$ do
8. $C \leftarrow M[i][k] + M[k+1][j] + P[i-1] \times P[k] \times P[j]$
- 9.
10. If $C < M[i][j]$ then.
11. $M[i][j] \leftarrow C$
11. return $M[1][n]$.

CODE:

```
#include <iostream>
#include <climits>
using namespace std;

int matrixChainMultiplication(int p[], int n)
{
    int m[n][n];

    // Cost of multiplying one matrix = 0
    for (int i = 1; i < n; i++)
        m[i][i] = 0;

    // L = length of matrix chain
    for (int L = 2; L < n; L++)
    {
        for (int i = 1; i < n - L + 1; i++)
        {
            int j = i + L - 1;
            m[i][j] = INT_MAX;

            // Try every possible split
            for (int k = i; k < j; k++)
            {
                int cost = m[i][k]
                    + m[k + 1][j]
                    + p[i - 1] * p[k] * p[j];
                if (cost < m[i][j])
                    m[i][j] = cost;
            }
        }
    }

    return m[1][n - 1];
}

int main()
{
    // A1 = 10x20
    // A2 = 20x30
    // A3 = 30x40
    int p[] = {10, 20, 30, 40};

    int n = sizeof(p) / sizeof(p[0]);
```

```
cout << "Minimum number of multiplications = "  
    << matrixChainMultiplication(p, n);  
  
return 0;  
}
```

OUTPUT:

Output

Minimum number of multiplications = 18000

TIME COMPLEXITY:

Time complexity.

$$T(n) = \sum_{L=2}^n \sum_{i=1}^{n-L+1} (L-1).$$

$$T(n) = \underline{\underline{O(n^3)}}$$

SPACE COMPLEXITY:

Space complexity.

$$\frac{n(n-1)}{2} = \underline{\underline{O(n^2)}}$$

CONCLUSION:

Chain Matrix Multiplication using Dynamic Programming finds the optimal order of multiplying a chain of matrices with the minimum number of scalar multiplications. It avoids repeated calculations by storing previous results in a DP table. The algorithm has $O(n^3)$ time complexity and $O(n^2)$ space complexity.